

Topic: 3D Fireworks Simulation and Water Reflection System

Introduction:

- Implemented a 3D fireworks particle system with lake surface reflections
- Simulates fireworks launch, explosion, and fading stages
- Integrated camera control, lighting, and scene rendering for realism
- Uses HDR rendering to reflect fireworks dynamically on the lake surface

Implementation:

- Camera
 - WASD for camera movement, arrow keys for viewing direction

```
// camera 移動 (WASD)
if (glfwGetKey(window, GLFW_KEY_W) == GLFW_PRESS){
    Camera::move(Camera::front * cameraSpeed); // 向前
}
if (glfwGetKey(window, GLFW_KEY_S) == GLFW_PRESS){
    Camera::move(-Camera::front * cameraSpeed); // 向后
}
if (glfwGetKey(window, GLFW_KEY_A) == GLFW_PRESS){
    Camera::move(-glm::normalize(glm::cross(Camera::front, Camera::up)) * cameraSpeed); // 向左
}
if (glfwGetKey(window, GLFW_KEY_D) == GLFW_PRESS){
    Camera::move(glm::normalize(glm::cross(Camera::front, Camera::up)) * cameraSpeed); // 向右
}

// 視角調整 (方向鍵)
const float rotationSpeed = 100.0f * deltaTime; // 旋轉速度 (加快)
if (glfwGetKey(window, GLFW_KEY_UP) == GLFW_PRESS){
    Camera::adjustPitch(rotationSpeed); // 向上看
}
if (glfwGetKey(window, GLFW_KEY_DOWN) == GLFW_PRESS){
    Camera::adjustPitch(-rotationSpeed); // 向下看
}
if (glfwGetKey(window, GLFW_KEY_LEFT) == GLFW_PRESS){
    Camera::adjustYaw(-rotationSpeed); // 向左看
}
if (glfwGetKey(window, GLFW_KEY_RIGHT) == GLFW_PRESS){
    Camera::adjustYaw(rotationSpeed); // 向右看
}
```

- Particle System
 - Launch stage: Firework particles are launched with initial upward velocity and rise until $v.y < 0.5$ m/s, at which point they enter the Burst stage and the original rocket particle is recycled.

```

if (!is_boomed) {
    if (v.y < 0.5f) {
        is_boomed = true;
        this->recycle = true;

        Light explosionLight;
        explosionLight.position = loc;
        explosionLight.color = color;
        explosionLight.intensity = 600.0f; // 初始强度
        explosionLight.ttl = 5.0f;        // 持續時間 (秒)
        lights.push_back(explosionLight);
    }
}

```

- Burst stage: 200–400 explosion sub-particles are generated at the explosion point and corresponding light source for each sub-particle is created to simulate the explosion flash. Each particle is assigned an initial velocity and TTL, and they spread outward uniformly in a spherical pattern.

```

for (int i = 0; i < 200; ++i) {
    glm::vec3 dir = glm::normalize(glm::vec3(
        (rand() % 200 - 100) / 100.0f,
        (rand() % 200 - 100) / 100.0f,
        (rand() % 200 - 100) / 100.0f
    ));
    glm::vec3 velocity = dir * 3.0f;
    Particle p;
    p.initialize(loc, dir, velocity, color2, 1.0f, 5.0f,
        true, false, glm::vec3(0.0f, -0.981f, 0.0f));
    newParticles.push_back(p);
}
Light explosionLight1;
explosionLight1.position = loc;
explosionLight1.color = color1;
explosionLight1.intensity = 500.0f; // 初始强度
explosionLight1.ttl = 5.0f;        // 持續時間 (秒)

```

- Fade stage: As a particle's TTL decreases, its color and transparency gradually change, transitioning from bright to dim until it becomes completely transparent. Once the lifetime ends, the particle is no longer rendered.

```

particles.erase(
    std::remove_if(
        particles.begin(),
        particles.end(),
        [](const Particle &p) { return p.check_recycle(); }
    ),
    particles.end()
);

```

- Trailing Effect: Each non-trailing particle generates a trailing particle every frame with zero velocity and acceleration at current position. Trailing particles fade rapidly to simulate visual persistence, enhancing the overall visual effect.

```
if(!is_tail){ // 在原地添加一个粒子實現拖尾
    Particle tail;
    tail.initialize(loc, glm::vec3(0.0f, 0.0f, 0.0f),
                    glm::vec3(0.0f, 0.0f, 0.0f), color,
                    transparency-0.2f, 3.0f, true, true,
                    glm::vec3(0.0f, 0.0f, 0.0f));
    newParticles.push_back(tail);
}
```

- Particle Lifetime (TTL) & Transparency Control: In Launch stage, transparency of non-trailing particles gradually decreases at 0.2 per second to keep the fireworks clearly visible as they rise. In Burst stage, TTL is actively decremented and transparency fades faster at 0.3 per second.

```
transparency = std::max(0.0f, transparency - 0.2f * deltaTime);
} else {
    ttl -= deltaTime;
    transparency = std::max(0.0f, transparency - 0.3f * deltaTime);
}
```

- Basic Rendering & Blending:
 - Billboard Rendering: Each particle is rendered as a quad that always faces the camera, avoiding drawing actual 3D geometry.

```
// 速度對齊模式：粒子沿速度方向拉伸
vec3 velocityDir = normalize(instanceVel);

// 計算速度在螢幕空間的投影方向
// upDir 沿速度方向（拉伸方向）
// rightDir 垂直於速度且在視圖平面上

// 使用相機的 right 向量來計算垂直於速度的方向
vec3 sideDir = normalize(cross(velocityDir, cameraRight));
if (length(cross(velocityDir, cameraRight)) < 0.01) {
    // 如果速度接近平行於 cameraRight，使用 cameraUp
    sideDir = normalize(cross(velocityDir, cameraUp));
}

// 重新計算確保正交
rightDir = normalize(cross(velocityDir, sideDir));
upDir = velocityDir;
```

- Gaussian fall-off: Produce soft glow and smooth edges, simulating the appearance of a projected 3D Gaussian blob.

```
// === Gaussian Fall-off (高斯衰減) ===
// 使用多層高斯函數創造發光效果
float sigma1 = 0.1; // 核心 - 很集中
float sigma2 = 0.3; // 中層光暈
float sigma3 = 0.5; // 外層柔和光暈

// 高斯函數:  $\exp(-dist^2 / (2 * sigma^2))$ 
float core =  $\exp(-dist * dist / (2.0 * sigma1 * sigma1))$ ;
float mid =  $\exp(-dist * dist / (2.0 * sigma2 * sigma2))$ ;
float outer =  $\exp(-dist * dist / (2.0 * sigma3 * sigma3))$ ;
```

- Additive blending to accumulate brightness when particles overlap.

```
// 設定渲染狀態
glEnable(GL_BLEND);
// Additive Blending: 重疊的粒子會疊加變亮，產生發光效果
glBlendFunc(GL_SRC_ALPHA, GL_ONE);
glDepthMask(GL_FALSE); // 禁用深度寫入，但保留深度測試
```

- Scene Setting
 - Skybox:
 - Rendered as a spherical shell surrounding the scene
 - Sphere vertices generated via latitude-longitude parameterization
 - Mesh data uploaded using VAO / VBO / EBO
 - Cubemap texture sampled using direction vectors to render the sky background

```
// 生成球體網格
const int stacks = 32; // 緯度分段
const int slices = 64; // 經度分段

std::vector<float> vertices;
std::vector<unsigned int> indices;

// 生成球體頂點
for (int i = 0; i <= stacks; ++i) {
    float phi = M_PI * float(i) / float(stacks); // 0 到  $\pi$ 
    float y = cos(phi) * skybox_size;
    float r = sin(phi) * skybox_size;

    for (int j = 0; j <= slices; ++j) {
        float theta = 2.0f * M_PI * float(j) / float(slices); // 0 到  $2\pi$ 
        float x = r * cos(theta);
        float z = r * sin(theta);

        vertices.push_back(x);
        vertices.push_back(y);
        vertices.push_back(z);
    }
}
```

```
glGenVertexArrays(1, &skyboxVAO);
glGenBuffers(1, &skyboxVBO);
glGenBuffers(1, &skyboxEBO);

glBindVertexArray(skyboxVAO);

glBindBuffer(GL_ARRAY_BUFFER, skyboxVBO);
glBufferData(GL_ARRAY_BUFFER, vertices.size() * sizeof(float), vertices.data(), GL_STATIC_DRAW);

glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, skyboxEBO);
glBufferData(GL_ELEMENT_ARRAY_BUFFER, indices.size() * sizeof(unsigned int), indices.data(), GL_STATIC_DRAW);

glEnableVertexAttribArray(0);
glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 3 * sizeof(float), (void*)0);

glBindVertexArray(0);
```

- Tree:
 - Random polar-coordinate distribution
 - Angle $\in [0, 2\pi]$, radius with power-law bias ($\exp = 0.7$)
 - 4–6 blob-based radial deformations for irregular silhouettes
 - 4–6 vertical layers with procedural noise and random offsets
 - Rendered as instanced billboards
 - Rendering order:
 - Trunk and branches first
 - Alpha blending enabled, leaves drawn in one instanced draw call

```
// 使用極坐標生成圓形分布，添加不規則性
float angle = randomFloat(0, 2 * 3.14159f);

// 根據角度計算不規則半徑（使用blob來創建不規則形狀）
float irregularRadius = radius;
for (const auto& blob : blobs) {
    float angleDiff = angle - blob.angle;
    // 使用柔和的權重函數
    float weight = (cos(angleDiff) + 1.0f) * 0.5f;
    weight = pow(weight, 2.0f); // 平方使過渡更柔和
    irregularRadius += (blob.radiusScale - 1.0f) * radius * 0.3f * weight;
}
```

```
// ===== 渲染葉子 Billboard =====
glUseProgram(leafShaderProgram);

glUniformMatrix4fv(glGetUniformLocation(leafShaderProgram, "model"), 1, GL_FALSE, &model[0][0]);
glUniformMatrix4fv(glGetUniformLocation(leafShaderProgram, "view"), 1, GL_FALSE, &view[0][0]);
glUniformMatrix4fv(glGetUniformLocation(leafShaderProgram, "projection"), 1, GL_FALSE, &projection[0][0]);
glUniform3fv(glGetUniformLocation(leafShaderProgram, "viewPos"), 1, &viewPos[0]);

// 啟用混合以支持透明度
glEnable(GL_BLEND);
glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
glBindVertexArray(leafVAO);
glDrawElementsInstanced(GL_TRIANGLES, 6, GL_UNSIGNED_INT, 0, leafInstanceCount);
glBindVertexArray(0);
glDisable(GL_BLEND);
```

- Mountain:

- Using a triangle mesh geometry
- Surface color is determined based on world-space height, where lower regions use rock textures and higher regions gradually blend into snow, creating natural elevation-based terrain layers
- Both rock and snow surfaces use procedural bump-like normal perturbations to produce shading variations across slopes, enhancing the three-dimensional form of the mountain

```
glGenVertexArrays(1, &mountainVAO);
glGenBuffers(1, &mountainVBO);
glGenBuffers(1, &mountainEBO);

glBindVertexArray(mountainVAO);
glBindBuffer(GL_ARRAY_BUFFER, mountainVBO);
glBufferData(GL_ARRAY_BUFFER, vertices.size() * sizeof(float), vertices.data(), GL_STATIC_DRAW);
```

```
// ===== 雪的微起伏法線 (假 bump) =====
// 用 fbm 在 xz 平面做一個「雪高度場」，用有限差分估 gradient 來造法線
float snowFreq = 0.18;      // 起伏的「大小」(越小越大塊，越大越碎)
float snowAmpN = 2.2;       // 起伏造成的法線強度(越大陰影越明顯)
float e = 0.8;              // 差分距離(世界座標，太小會抖/太大會糊)

float h0 = fbm(FragPos.xz * snowFreq);
float hx = fbm((FragPos.xz + vec2(e, 0.0)) * snowFreq);
float hz = fbm((FragPos.xz + vec2(0.0, e)) * snowFreq);

// dh/dx, dh/dz
float dhdx = (hx - h0) / e;
float dhdz = (hz - h0) / e;

// 雪的微法線：y 給 1，x/z 用斜率(乘上 snowAmpN 控制陰影強度)
vec3 N_snow = normalize(vec3(-dhdx * snowAmpN, 1.0, -dhdz * snowAmpN));
```

○ Lake:

- Surface is rendered as a flat polygonal mesh
- In the vertex stage, time-varying sine waves introduce small height perturbations to create gentle water motion
- In the fragment stage, a Fresnel effect based on the view direction and surface normal is applied
- Blend water color, lighting, specular highlights, and reflections together

```
// ---- 生成不規則形狀湖面 ----
// 中心點
vertices.push_back(0.0f);
vertices.push_back(waterLevel);
vertices.push_back(0.0f);
vertices.push_back(0.5f);
vertices.push_back(0.5f);
vertices.push_back(0.0f);
vertices.push_back(1.0f);
vertices.push_back(0.0f);

// 用噪聲函數產生不規則邊緣
auto noise = [](float angle) {
    // 多個正弦波疊加產生自然的不規則感
    float n = 1.0f;
    n += 0.15f * std::sin(angle * 2.0f + 0.5f); // 大
    n += 0.10f * std::sin(angle * 3.0f + 1.2f); // 中
    n += 0.08f * std::sin(angle * 5.0f + 2.7f); // 小
    n += 0.05f * std::sin(angle * 7.0f + 0.3f); // 細節
    n += 0.03f * std::sin(angle * 11.0f + 1.8f); // 更多細節
    return n;
};
```

```

glGenVertexArrays(1, &lakeVAO);
glGenBuffers(1, &lakeVBO);
glGenBuffers(1, &lakeEBO);

glBindVertexArray(lakeVAO);
glBindBuffer(GL_ARRAY_BUFFER, lakeVBO);
glBufferData(GL_ARRAY_BUFFER, vertices.size() * sizeof(float), vertices.data(), GL_STATIC_DRAW);

glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, lakeEBO);
glBufferData(GL_ELEMENT_ARRAY_BUFFER, indices.size() * sizeof(unsigned int), indices.data(), GL_STATIC_DRAW);

```

- Lake shore:
 - Rendered as a ground mesh surrounding the lake
 - Use height- and position-based variation to create a natural transition between land and water, while textures and procedural noise are applied to break uniformity and add terrain detail

```

// 內圈是湖邊（不規則形狀），外圈延伸到場景邊緣
const int segments = 128; // 與湖面相同
const float baseInnerRadius = 80.0f; // 配合放大的湖面
const float outerRadius = 500.0f; // 擴大陸地配合大天空盒
const float groundY = -0.15f; // 地面略低於水面，避免 Z-fighting

// 與湖面相同的噪聲函數產生配合的不規則邊緣
auto noise = [](float angle) {
    float n = 1.0f;
    n += 0.15f * std::sin(angle * 2.0f + 0.5f);
    n += 0.10f * std::sin(angle * 3.0f + 1.2f);
    n += 0.08f * std::sin(angle * 5.0f + 2.7f);
    n += 0.05f * std::sin(angle * 7.0f + 0.3f);
    n += 0.03f * std::sin(angle * 11.0f + 1.8f);
    return n;
};

```

- Shader
 - Firework Particle Render:
 - When fireworks explode, point light sources are created at the explosion positions.
 - These lights are stored in the lights array with color, intensity, and lifetime (TTL)
 - The dynamic lights are passed to the fragment shaders of static scene objects
 - Fragment shaders compute per-fragment lighting using distance attenuation

```

float calculateAttenuation(float distance) {
    float constant = 1.0;
    float linear = 0.10;
    float quadratic = 0.03;
    return 1.0 / (constant + linear * distance + quadratic * (distance * distance));
}

// ===== Add firework lighting (added) =====
vec3 fireworkAdd = vec3(0.0);

for (int i = 0; i < numLights && i < MAX_LIGHTS; ++i) {
    vec3 Ldir = lights[i].position - FragPos;
    float dist = length(Ldir);
    vec3 Lp = Ldir / max(dist, 1e-4);

    float att = calculateAttenuation(dist);

    // diffuse: 葉子可以稍微更吃光、樹幹少一點 (但不改原本材質)
    float ndl = max(dot(N_use, Lp), 0.0);
    float leafBoost = mix(0.65, 1.15, isLeaf);
    vec3 diffAdd = baseCol * ndl * leafBoost;

    // spec: 讓煙火在葉子上有一點亮點
    vec3 Hp = normalize(Lp + V);
    float specP = pow(max(dot(N_use, Hp), 0.0), mix(24.0, 48.0, isLeaf));
    vec3 specAdd = vec3(specP) * mix(0.03, 0.07, isLeaf);

    // 疊加: 強度縮放
    fireworkAdd += (diffAdd + specAdd) * lights[i].color * lights[i].intensity * att * 0.05;
}

// 加到原本 col, 不影響原本月光/材質
col += fireworkAdd;

```

- Water Surface Render:

- Fireworks are included in the reflection pass and rendered to reflTex
- The water shader samples the reflection texture using perspective-correct UVs
- Wave normals are computed from time-varying sine-wave height fields
 - Multiple wave directions and frequencies create dynamic water motion

```
// 加入輕微的波浪效果，邊緣處衰減
vec3 pos = aPos;
float wave1 = sin(pos.x * 0.5 + time * 2.0) * 0.02;
float wave2 = sin(pos.z * 0.3 + time * 1.5) * 0.02;
float wave3 = sin((pos.x + pos.z) * 0.4 + time * 2.5) * 0.015;
pos.y += (wave1 + wave2 + wave3) * edgeFade;

// 計算波浪後的法線，同樣邊緣衰減
float dx = (cos(pos.x * 0.5 + time * 2.0) * 0.5 * 0.02
           + cos((pos.x + pos.z) * 0.4 + time * 2.5) * 0.4 * 0.015) * edgeFade;
float dz = (cos(pos.z * 0.3 + time * 1.5) * 0.3 * 0.02
           + cos((pos.x + pos.z) * 0.4 + time * 2.5) * 0.4 * 0.015) * edgeFade;
vec3 waveNormal = normalize(vec3(-dx, 1.0, -dz));
```

- Reflection UVs are distorted by wave normals.

```
// 加入微小的法線扭曲，讓倒影有水波效果但保持清晰
vec2 distortion = N.xz * 0.02; // 很小的扭曲
reflUV += distortion;
reflUV = clamp(reflUV, 0.002, 0.998);
```

- High-intensity firework lights increase reflection strength and add a reflection boost

```
// 煙花爆炸時的特殊反射效果
vec3 fireworkReflection = vec3(0.0);
for (int i = 0; i < numLights && i < MAX_LIGHTS; ++i) {
    // 只有強度較高的光源（煙花爆炸）才產生明顯的水面反射
    if (lights[i].intensity > 130.0) {
        float distance = length(lights[i].position - FragPos);
        float reflectionIntensity = lights[i].intensity / (distance * distance + 1.0);

        // 反射點的位置（光源在水面下的鏡像）
        vec3 reflectedLightPos = lights[i].position;
        reflectedLightPos.y = -reflectedLightPos.y; // 假設水面在 y=0

        vec3 toReflection = normalize(reflectedLightPos - FragPos);
        float reflectionStrength = max(dot(V, toReflection), 0.0);
        reflectionStrength = pow(reflectionStrength, 2.0);

        fireworkReflection += lights[i].color * reflectionIntensity * reflectionStrength * 0.01;
    }
}
```

- Fresnel term (Schlick approximation) is applied
 - Reflections are stronger at grazing view angles
 - Reflections are weaker when viewed from above
 - Reflection strength is controlled by Fresnel and edge fading

```
// Fresnel: 越斜看越像鏡面 (更像水)
float ndv = clamp(dot(N, V), 0.0, 1.0);
float fresnel = pow(1.0 - ndv, 5.0);
fresnel = 0.05 + 0.95 * fresnel;
float fresnelMax = 0.90; // 提高反射上限，讓倒影更明顯
fresnel = min(fresnel, fresnelMax);
float grazingDamp = 0.75 + 0.25 * ndv; // 提高基礎反射
fresnel *= grazingDamp;

// 基礎水色：做深淺，往下看更深更暗，斜看更亮
vec3 shallow = waterColor * vec3(0.95, 1.0, 1.0); // 淺水稍亮
vec3 deep     = waterColor * vec3(0.15, 0.25, 0.42); // 深水偏暗
vec3 baseColor = mix(shallow, deep, ndv);
vec3 tint = texture(waterTexture, TexCoords).rgb;
baseColor *= mix(vec3(1.0), tint, 0.6);
```

Problems encountered:

1. Water surface appears mirror-like

○ Before

- The water surface reflects too much light, making the reflected color unlike that of the water at night.
- The light from the explosion of fireworks turns into a burst of white light, obscuring the reflection of the fireworks particles.



○ Fix

- Adjust / Add parameters in water.frag
- Separate the shading into two states and use different parameter sets for each:
 - With fireworks: stronger reflection and higher perceived brightness due to reflected firework light.
 - Without fireworks: weaker reflection and overall darker appearance.
- Darken the base water color

```

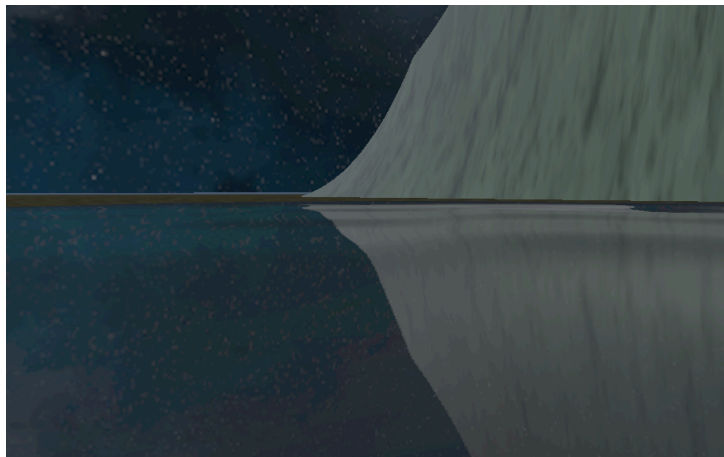
float ambK = hasFirework ? 0.01 : 0.08;
float difK = hasFirework ? 0.05 : 0.35;

// 環境光 (提升亮度)
ambient += ambK * lights[i].color * lights[i].intensity * attenuation;

// 漫反射 (提升亮度)
float diff = max(dot(N, L), 0.0);
diffuse += difK * diff * lights[i].color * lights[i].intensity * attenuation;

// 鏡面反射 (水面高光)
float spec = pow(max(dot(N, H), 0.0), 192.0);
specular += 2.6 * spec * lights[i].color * lights[i].intensity * attenuation;

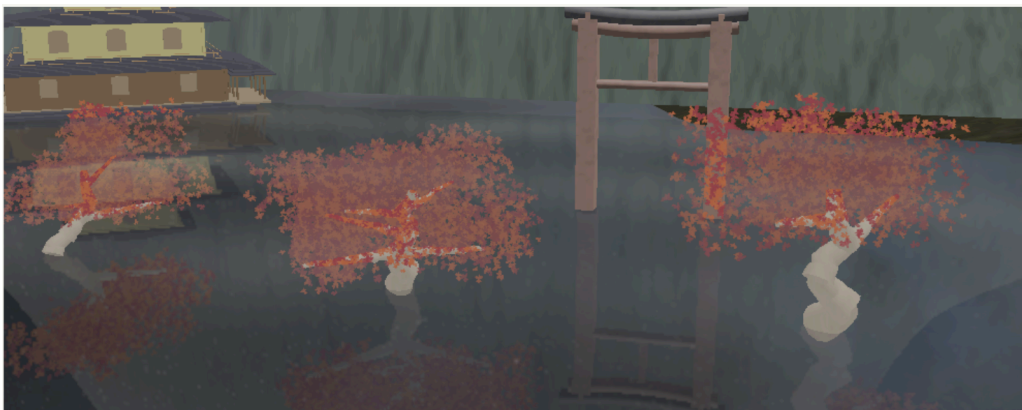
```



2. Depth & Transparency

○ Before

- Depth write was disabled for semi-transparent tree leaves
- Leaves did not write to the depth buffer, causing the lake (rendered later) to incorrectly occlude trees and cut off reflections



○ Fix

- Enable depth writing for tree leaves while keeping alpha blending

- Correct rendering order: opaque objects → leaves (depth write ON) → water (depth write OFF)

